ASSIGNMENT 04
From Daily CSVs to Automated ML Pipelines

MONTH 02 • DATA ENGINEERING • INDIVIDUAL ASSIGNMENT

Build a reliable delivery-delay prediction workflow using daily CSV files, Medallion Architecture, Apache Airflow, Docker, and Scikit-learn.

> **MAIN GOAL** • We are assessing how well you build reproducible, date-aware pipelines - not who gets the highest ML accuracy.

STAGE 01 / DATA ENGINEERING

Daily data to Gold
Ingest, clean, validate, and prepare ML-ready data.

STAGE 02 / ORCHESTRATION

Gold to trained model
Run, monitor, evaluate, and persist the model.

01 | Dataset and business scope

Use the supplied **46 daily orders CSV files** (2026-10-01 to 2026-11-15), sharing the same 15-column schema. Predict **late_delivery** (0 = on time, 1 = late). This is a standalone project; do not reuse old assignments or change the original raw files.

02 | Stage A - Explore and assess the data

- Inspect at least **three dated CSV batches** in Pandas; identify feature types, the label, and identifier/date fields.
- Measure missing values, invalid values, duplicate orders, and class distribution; record findings in the README.
- Explain which problems are fixed in Silver and which learned preprocessing steps belong inside ML training.

03 | Stage B - Build Bronze, Silver, and Gold

- **Bronze:** copy the selected daily CSV unchanged, preserving even bad records and duplicates.
- **Silver:** deduplicate `order_id`, standardize categories and types, and handle invalid or missing values with documented rules.
- **Gold:** prepare suitable prediction features, engineer at least **two** useful features, and keep the binary target.
- **Validation:** fail clearly on empty data, missing fields, duplicate IDs, incorrect dates, or invalid labels.

04 | Required data-processing DAG

extract_to_bronze → transform_to_silver

prepare_gold → validate_gold

Create **data_pipeline** with daily scheduling, at least 2 retries, one source file per logical processing date, and idempotent per-date outputs.

05 | ML approach and leakage prevention

Train a suitable Scikit-learn classification model using validated Gold data. Split train/test appropriately, fit learned preprocessing **only on training data**, report Accuracy / Precision / Recall / F1, and save the fitted model with Joblib.

load_gold_data → train_model → evaluate_model → save_model

Create a second Airflow DAG named **ml_training_pipeline**; manual triggering after validated Gold data is available is acceptable.

STRUCTURE | Project Files & Responsibilities

Use the provided student starter as your baseline. Complete the Python functions and DAGs yourself; generated outputs and evidence belong in the same repository.

06 | Required project structure

```
assignment04-delivery-delay/
|-- dags/
|   |-- data_pipeline.py
|   `-- ml_training_pipeline.py
|-- src/
|   |-- etl.py
|   `-- train.py
|-- data/
|   |-- raw/           # 46 daily CSV files
|   |-- bronze/
|   |-- silver/
|   |-- gold/
|   `-- ml/            # optional intermediates
|-- models/
|-- metrics/
|-- screenshots/
|-- Dockerfile
|-- docker-compose.yml
|-- requirements.txt
|-- .env.example
|-- data_dictionary.csv
`-- README.md
```

07 | What each core component must do

data/raw/	Keep all 46 source CSVs unchanged; choose files by batch date.
src/etl.py	Implement Bronze, Silver, Gold, and validation as reusable dated functions.
dags/data_pipeline.py	Declare and schedule the four dependent data-processing tasks.
src/train.py	Load validated Gold data; split, preprocess, train, evaluate, and save.
dags/ml_training_pipeline.py	Orchestrate load_gold_data, train_model, evaluate_model, save_model.
data/bronze, silver, gold	Write stable outputs per batch date; avoid duplicate or appended rows.
models/ + metrics/	Store the evaluated Joblib model and metrics.json as separate artifacts.
Dockerfile + compose	Run Airflow with access to code, inputs, outputs, logs, and artifacts.
screenshots/ + README	Show UI evidence and explain how another student can reproduce the runs.

DELIVERY | Engineering & Submission Requirements

08 | Airflow DAGs and ML deliverables

- **Data DAG:** schedule daily, use the run's processing date, set at least two retries, and preserve one date per DAG run.
- **ML DAG:** implement `load_gold_data → train_model → evaluate_model → save_model`; manual triggering after valid Gold output is acceptable.
- **ML artifacts:** save `models/delivery_delay_model.joblib` and `metrics/metrics.json`; include your split strategy and record counts.

09 | Reliability and Airflow monitoring

- **Retries:** set `retries=2`; provide safe evidence of a temporary task failure and successful retry.
- **Idempotency:** rerun the **same batch date twice**; show unchanged row counts and no duplicate `order_id` records.
- **Backfill:** process **three missed daily batches** (suggested: October 18-20, 2026) and show three corresponding Gold partitions.
- **Monitoring:** capture the Airflow DAG Graph, task states, and execution logs for both pipelines.

10 | Environment and required evidence

Use **Docker Compose** to launch Airflow, with access to data, DAGs, Python modules, models, and metrics. Keep these items in the submitted repository:

Code + config	DAG files, reusable Python modules, Dockerfile, docker-compose.yml, requirements.txt.
Layer examples	Generated Bronze / Silver / Gold CSV outputs for at least three distinct dates.
Model + metrics	Saved Joblib artifact, JSON evaluation metrics, and short explanation of features.
Screenshots	Successful Graph views, logs, retry proof, same-date rerun, and three-date backfill.

11 | GitHub submission and final checks

- Create **one new GitHub repository** for this assignment; do not submit your previous ML project.
- Write a README explaining the scenario, dataset, data-quality rules, the two DAGs, model choices, setup commands, and results.
- Push complete code and sample outputs; do not commit secrets, virtual environments, or unnecessary generated files.

Final acceptance check: a reviewer can start Airflow, run the dated data DAG, trigger ML training, open the model and metrics, and verify retry, rerun, and backfill evidence.

> **OPTIONAL BONUS** • Use PostgreSQL for processed data; automatically trigger ML training after valid Gold updates; add ETL tests or model versioning.